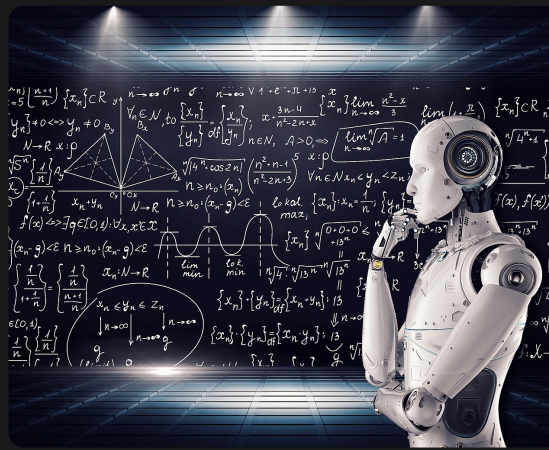




EUROPEAN
SPALLATION
SOURCE



WHAT AI CAN CHANGE IN CRYSTALLOGRAPHIC COMPUTING

Early lessons from building
a diffraction engine from scratch

Andrew Sazonov
ESS DMSC

OUTLINE

💡 Motivation

🏗️ What was built

🕒 Speed and correctness

🔧 How it was built

OUTLINE

💡 Motivation

🏗️ What was built

🕒 Speed and correctness

🔧 How it was built

AI IN SCIENTIFIC SOFTWARE



27th Congress and General Assembly
of the International Union of Crystallography

≈ 1,800 attendees

AI/ML named in the **title or abstract**

23

of 142 microsymposia

121

of 839 microsymposium talks

69

of 932 posters

HOW AI ENTERS SOFTWARE DEVELOPMENT

① AI writes all the code

small projects · a native CIF parser for CCTBX

Is it fast?

1.5 - 41×

faster than the old parser

same Python data
structures,
single keyword to switch

1.3 - 1.5×

faster than Gemmi

native path, matched
output;
5-27% less peak memory

8.1 s

for a 3.3 GB file

file sizes that were
previously
impossible become routine

Benchmarks as a decision instrument:

- Does the added code complexity pay for itself in speed?
- Does an optimisation help everywhere, or only on typical files?

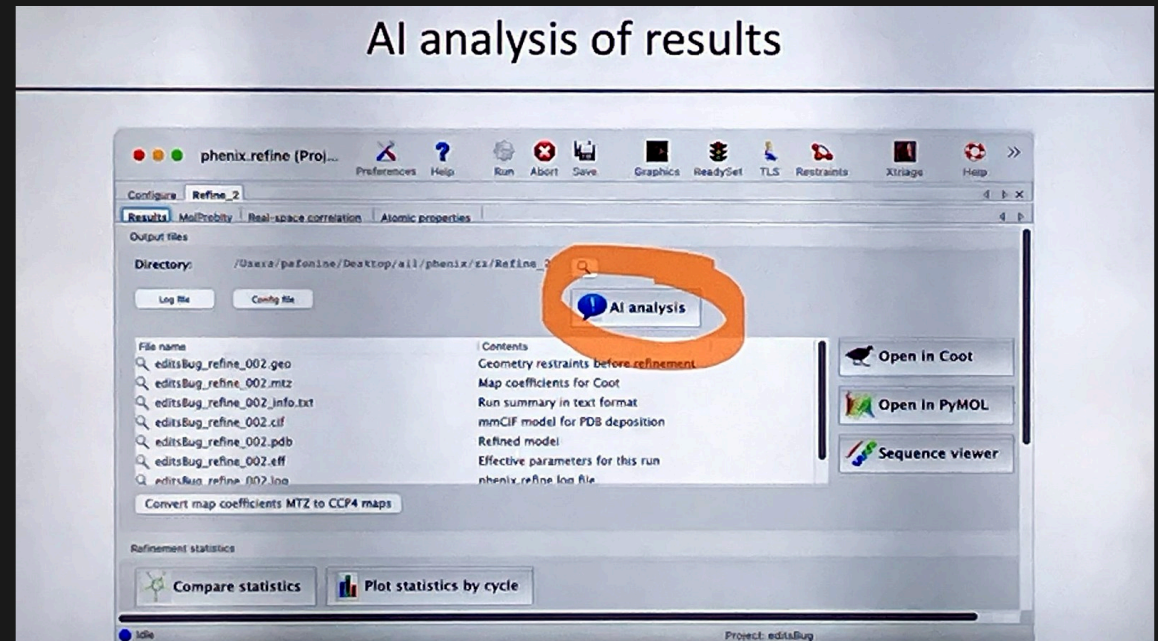
HOW AI ENTERS SOFTWARE DEVELOPMENT

① AI writes all the code

small projects · a native CIF parser for CCTBX

② AI assistants and chatbots

mature product · Phenix



HOW AI ENTERS SOFTWARE DEVELOPMENT

① AI writes all the code

small projects · a native CIF parser for CCTBX

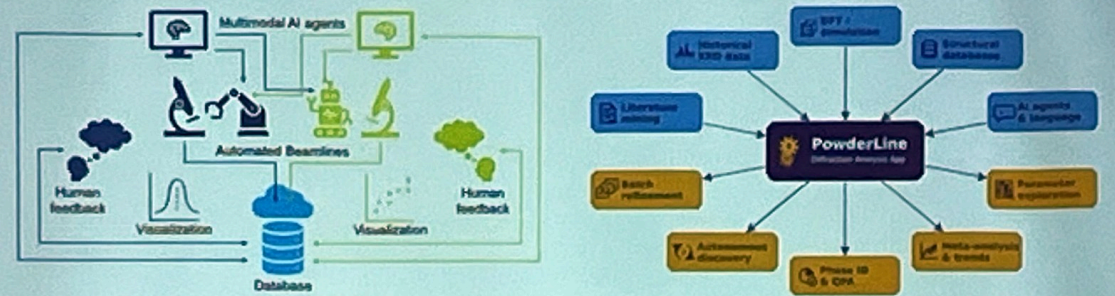
② AI assistants and chatbots

mature product · Phenix

③ AI orchestration

decides what to measure next, then processes it · NSLS-II

Summary and looking forward



- At NSLS-II we have developed a modular framework that enables collaborative human-AI operating modes on Bluesky-enabled instruments
- PowderLine transforms highly manual, pathway-dependent diffraction analyses into automatable, scalable, and reproducible tasks
- Together, these tools and AI-enhanced workflows enable a paradigm shift toward hypothesis driven autonomous experiments and accelerated discovery

HOW AI ENTERS SOFTWARE DEVELOPMENT

① AI writes all the code

small projects · a native CIF parser for CCTBX

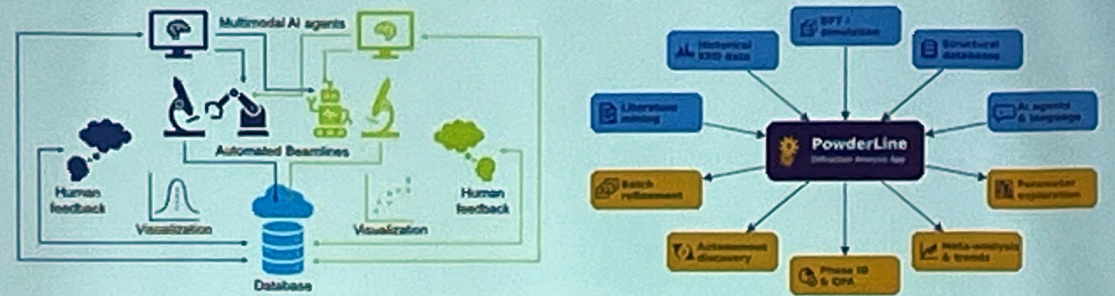
② AI assistants and chatbots

mature product · Phenix

③ AI orchestration

decides what to measure next, then processes it · NSLS-II

Summary and looking forward



- At NSLS-II we have developed a modular framework that enables collaborative human-AI operating modes on Bluesky-enabled instruments
- PowderLine transforms highly manual, pathway-dependent diffraction analyses into automatable, scalable, and reproducible tasks
- Together, these tools and AI-enhanced workflows enable a paradigm shift toward hypothesis driven autonomous experiments and accelerated discovery

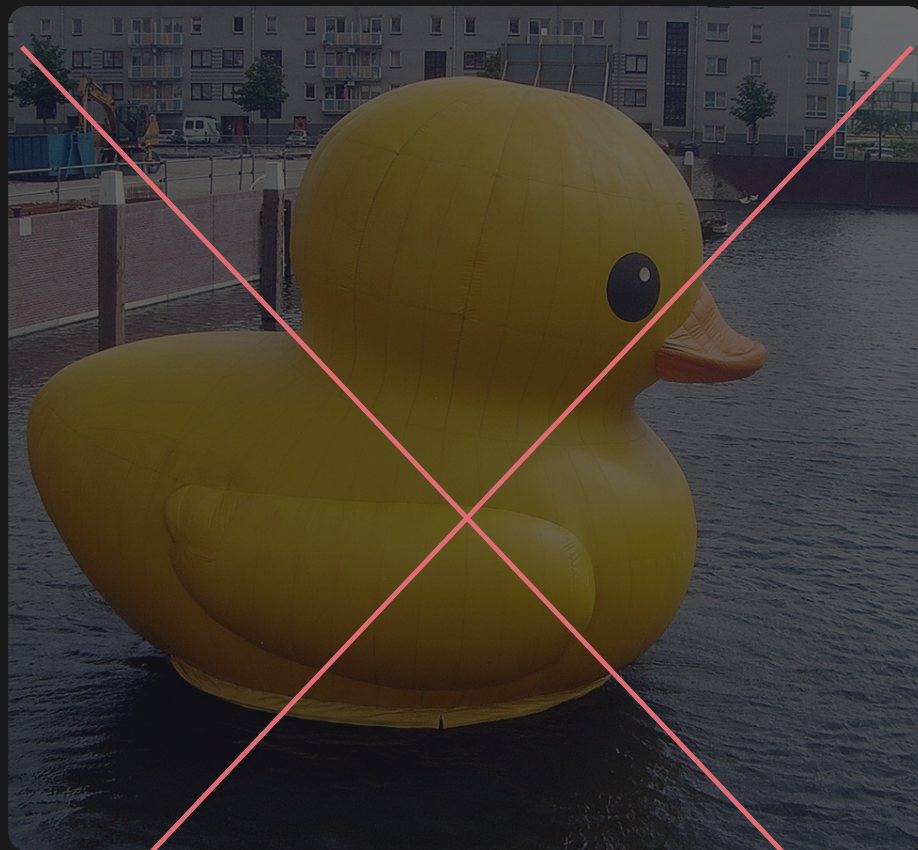
This talk focuses on ①

WHY?



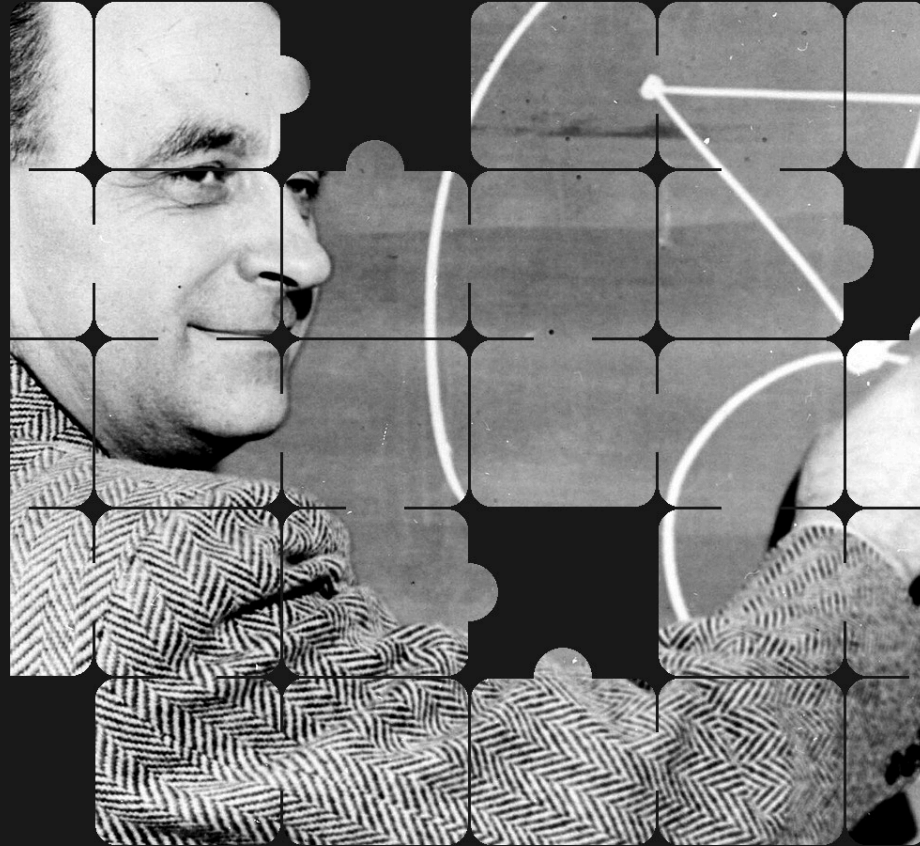
Just for fun

WHY?



Just for fun

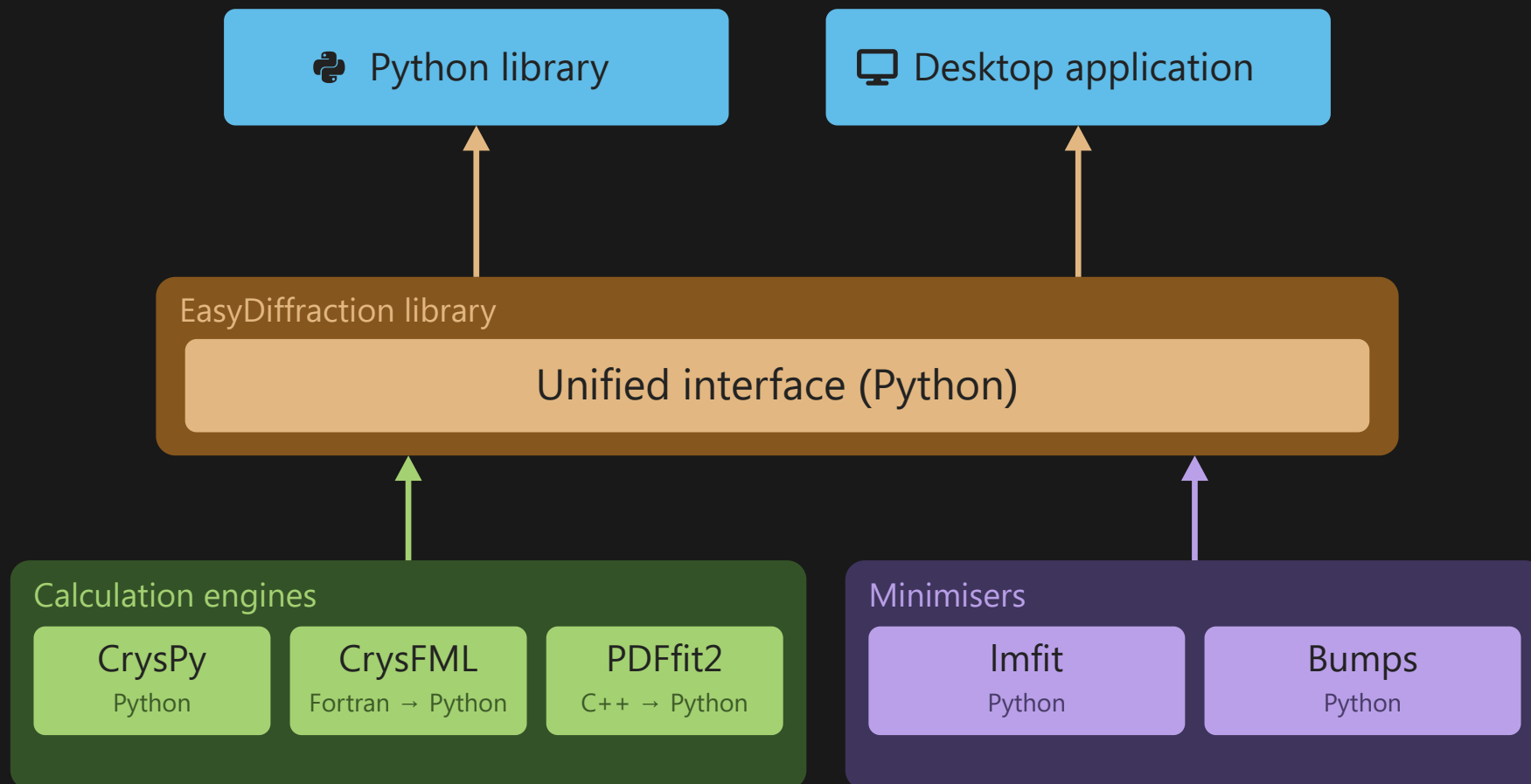
WHY?



To fill gaps in
the existing approach

THE EXISTING APPROACH

Reuse third-party calculation and fitting engines



WHAT IT COSTS US



Maintenance

wrappers and bindings for many libraries



Performance

overhead, non-optimised fitting



Extensibility

new features may require changes in several libraries

WHAT MODERN ANALYSIS NEEDS

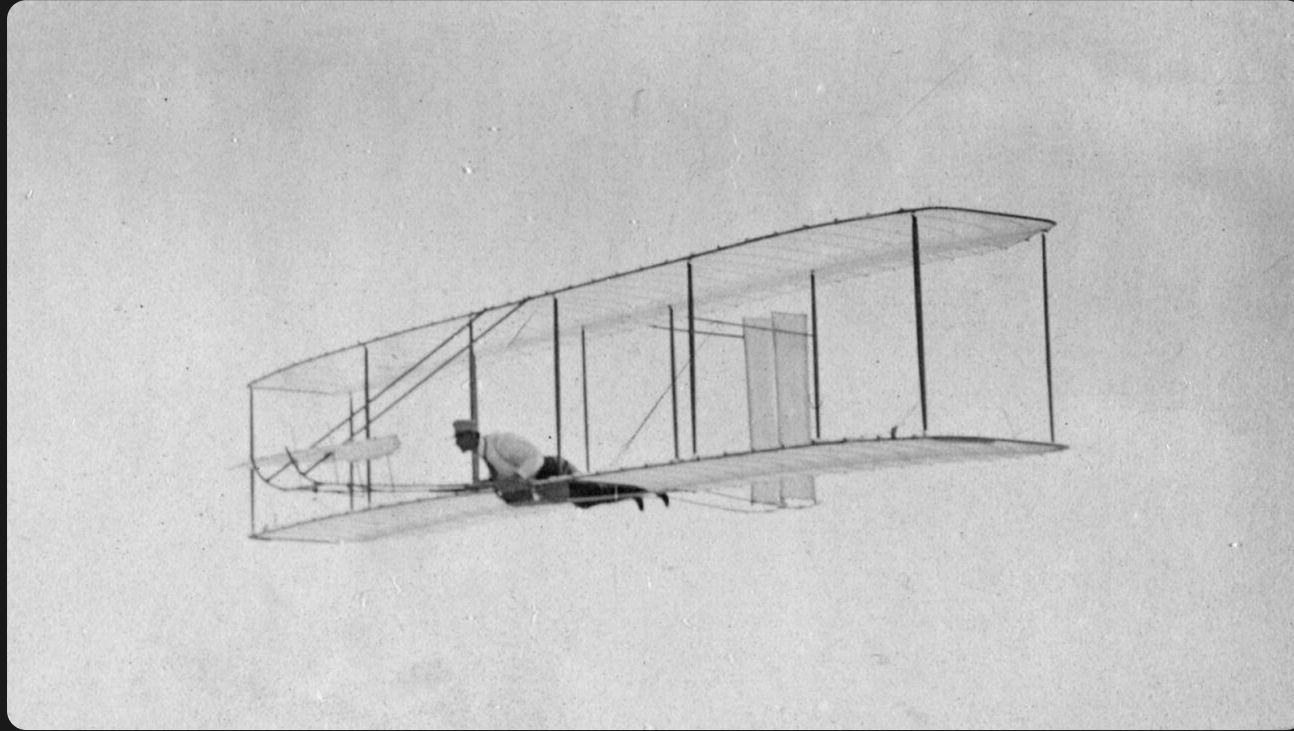
 **Large series** · up to 1000 datasets × 100 patterns each
temperature, field and time series from one experiment

 **Multidimensional data** and very large parameter spaces
2D and 3D detectors, joint refinement of many patterns at once

 **Automation** · experiments that run themselves
sequencing and recovery without a human · a bad dataset must not stop the run

As the data grows, **performance matters more and more**

THE EXPERIMENT



How far can we get if **AI writes every line**
of a real crystallographic engine?

OUTLINE

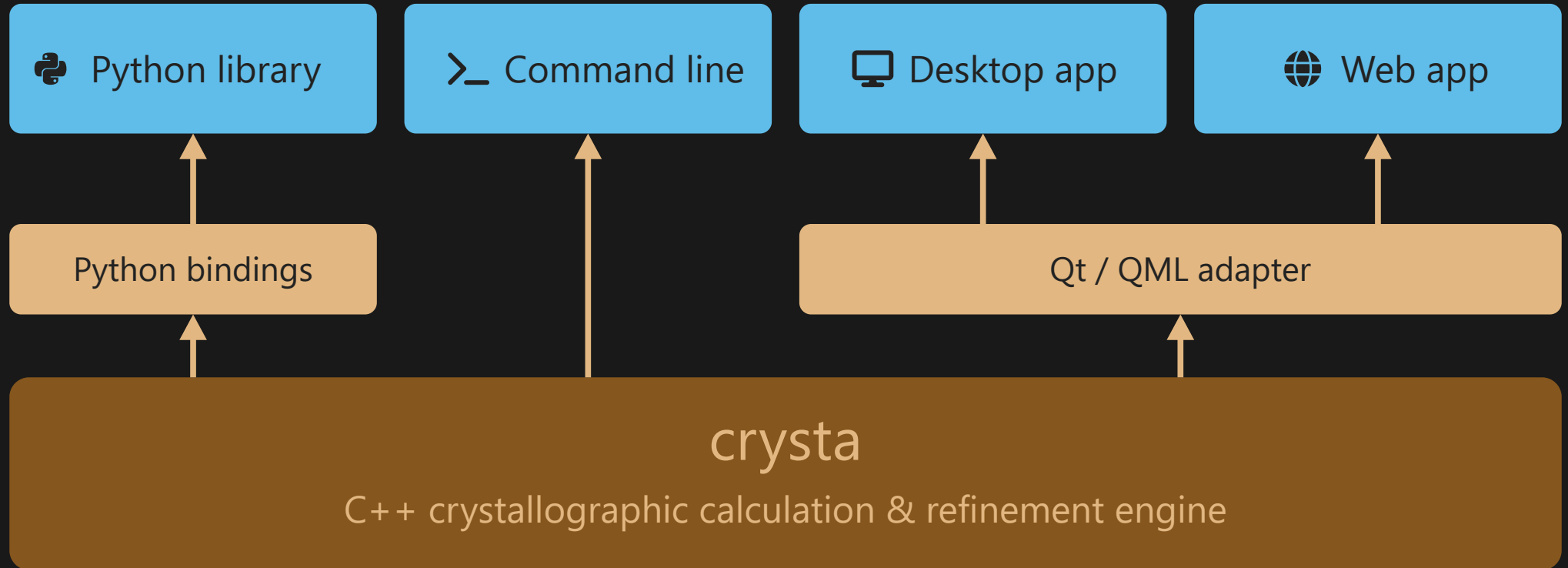
💡 Motivation

🏗️ What was built

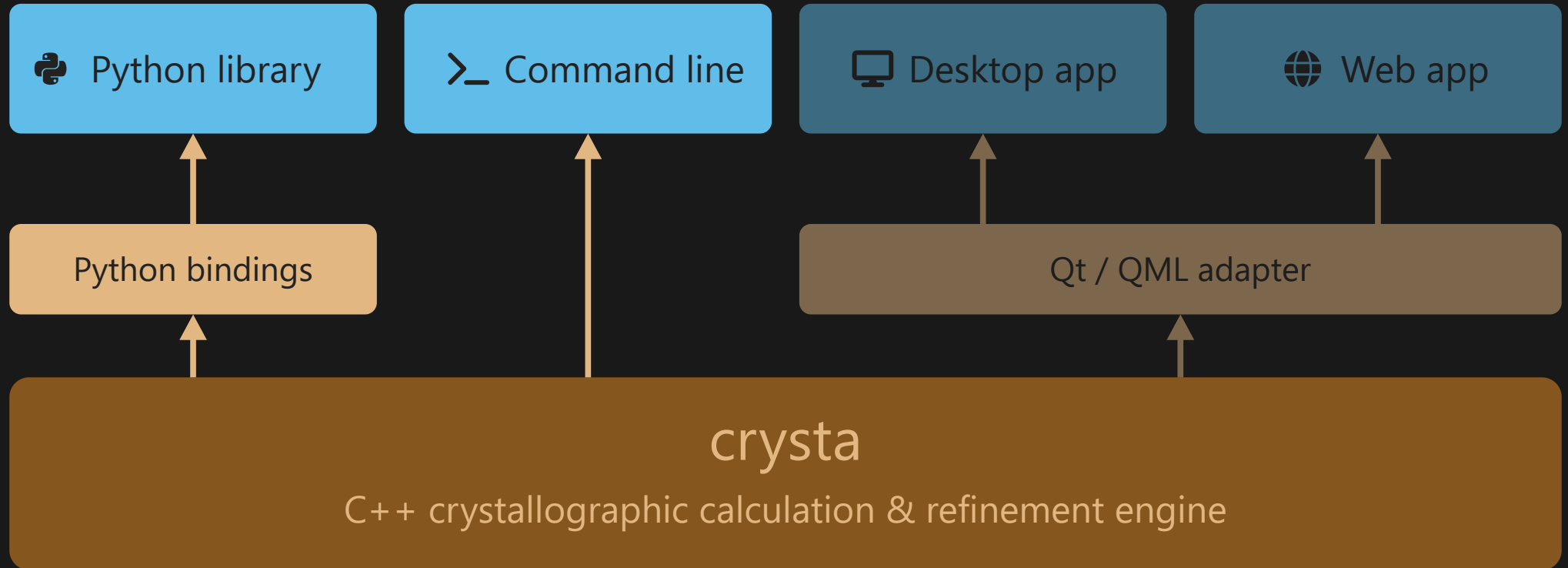
🕒 Speed and correctness

🔧 How it was built

ONE ENGINE, FOUR INTERFACES



ONE ENGINE, FOUR INTERFACES



WHAT IT STANDS ON

In the product

Eigen

linear algebra · the only one required

OpenMP

threads · optional — else it runs serially

SLEEFPipe

vector maths · optional — else one at a time

nanobind

the Python bindings · native builds only

In development

CMake · Ninja · scikit-build-core

configure, build and package the wheels

pixi

the environment, pinned for everyone

doctest · pytest

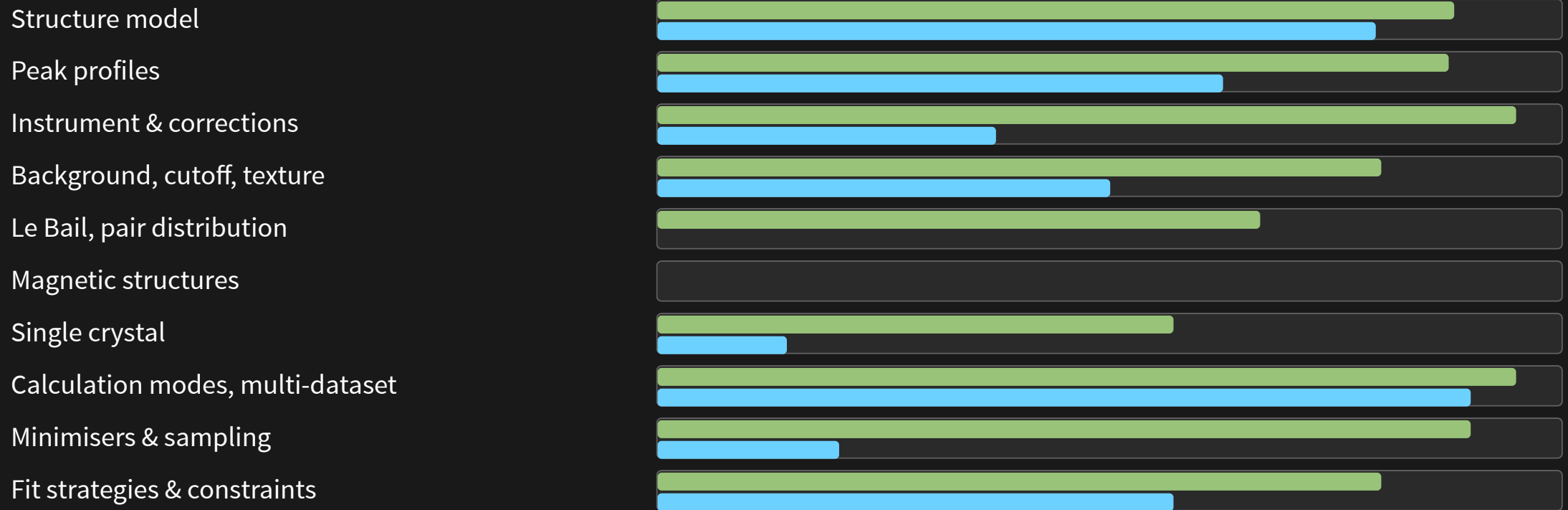
the tests, C++ and Python

ruff · clang-tidy · cppcheck · mypy

static analysis

HOW MUCH IS DONE

relative coverage from feature lists and my estimates ■ EasyDiffraction today ■ crysta today



About **eight weeks** of development

OUTLINE

💡 Motivation

🏗️ What was built

🕒 Speed and correctness

🔧 How it was built

THE TEST CASE

⚙️ **Sample** · NCAF, $\text{Na}_2\text{Ca}_3\text{Al}_2\text{F}_{14}$

a standard reference powder · cubic $I2_13$, $a = 10.2503 \text{ \AA}$, six sites

📐 **Measurement** · WISH at ISIS, banks 2 + 9 merged

one pattern at $2\theta = 58.33^\circ$ · 3977 points, 627 peaks in the fitted range

⚙️ **Refinement** · 48 free parameters

coordinates, displacement, scale, zero, profile, background · same start for every engine

⚖️ **Reference** · FullProf CLI 8.40

fp2k · same model and instrument parameters · every difference is measured against it

🔧 **Machine** · VISA, an Intel Xeon node · 6 cores

runs use 1 or 6 threads, as labelled

SPEED: ONE PATTERN CALCULATION

🧪 NCAF

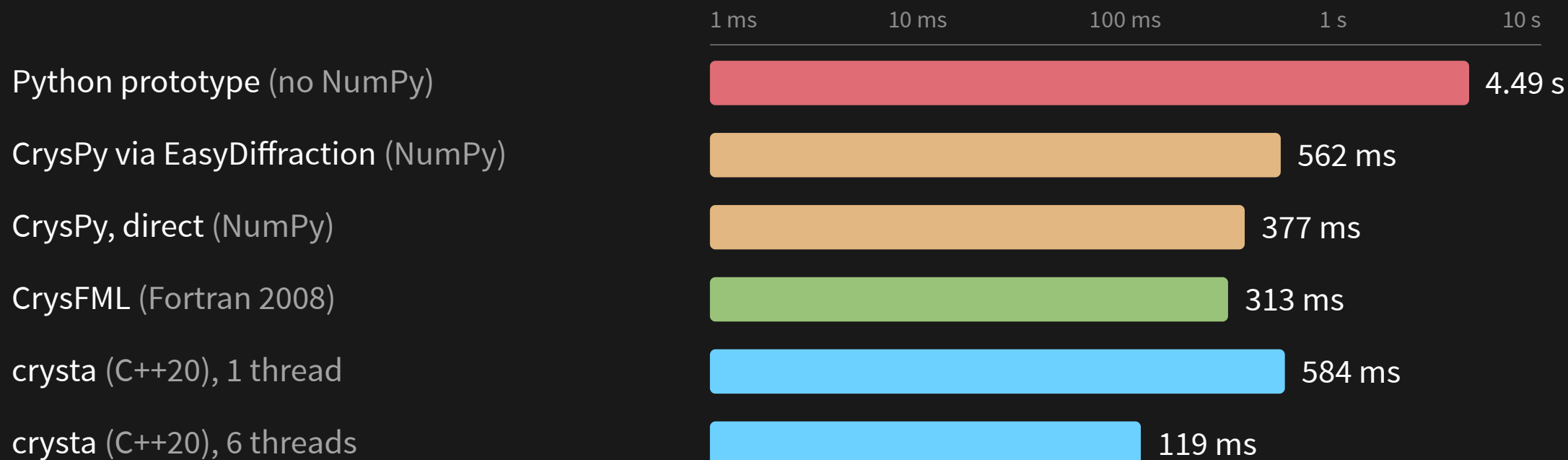
📈 3977 points

⬆️ 627 peaks

🔧 WISH 2+9

⚙️ VISA 6 cores

📊 log scale



SPEED: ONE PATTERN CALCULATION

🧪 NCAF

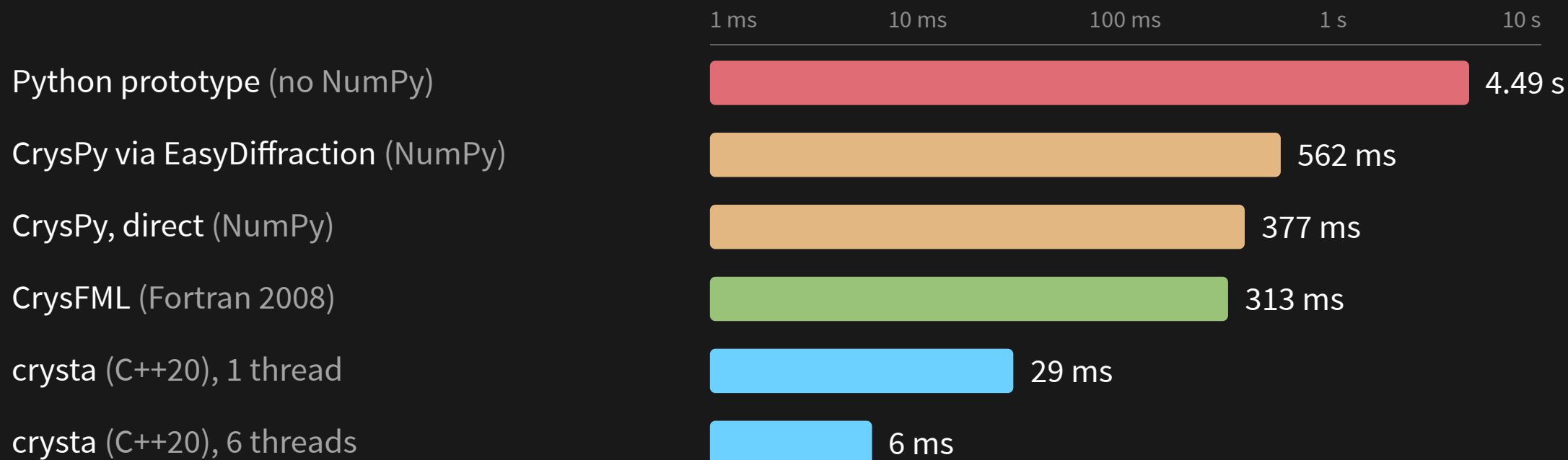
📈 3977 points

⬆️ 627 peaks

🔧 WISH 2+9

⚙️ VISA 6 cores

📊 log scale



✂️ Recomputing reflections that symmetry makes identical

AGREEMENT OF CALCULATED PATTERNS

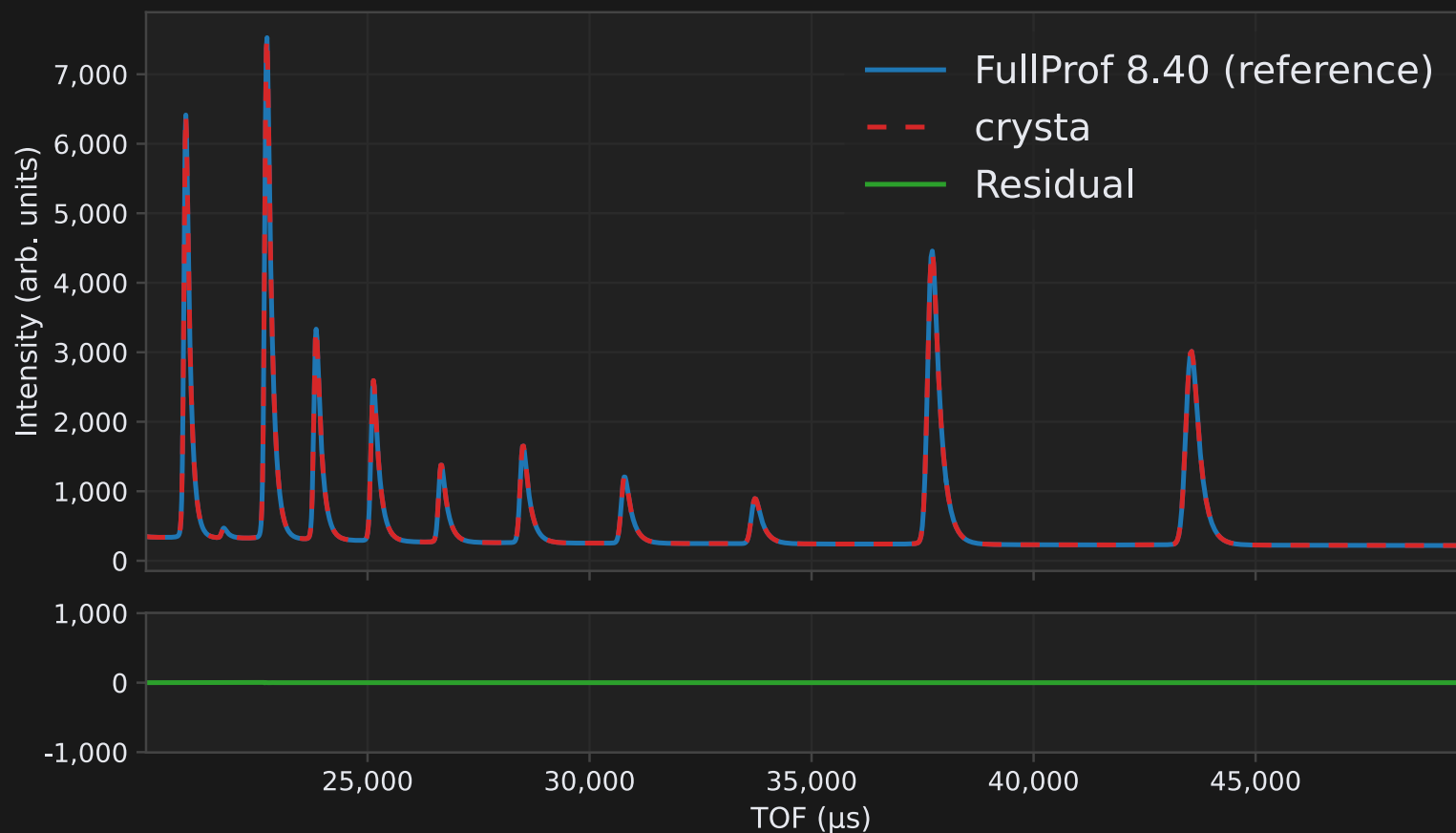
🧪 NCAF

📈 3977 points

📐 627 peaks

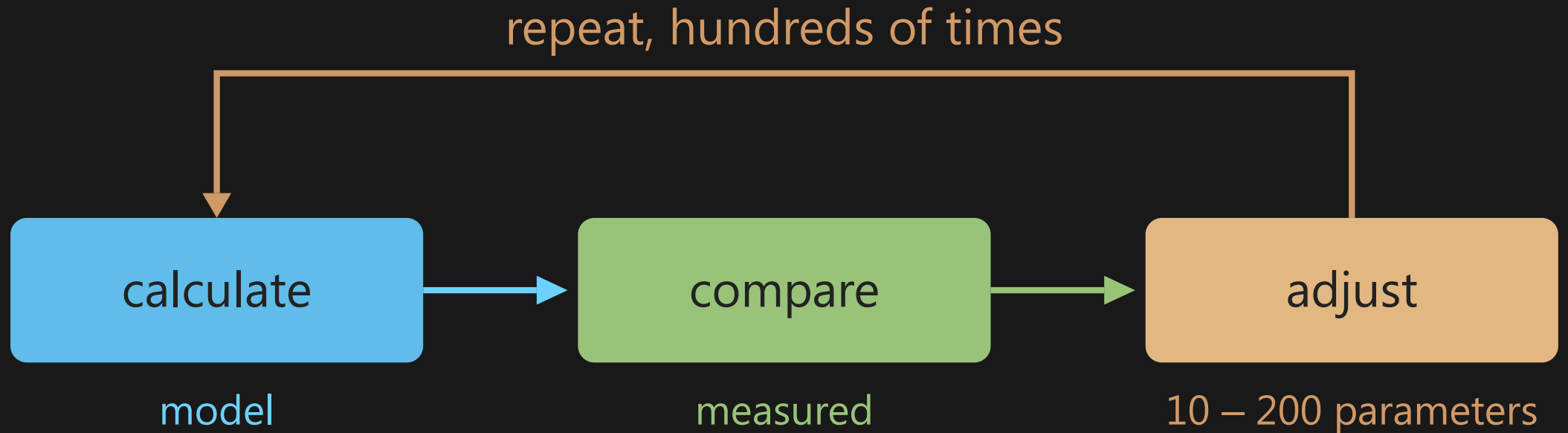
📡 WISH 2+9

💻 VISA 6 cores



Largest difference 3.9 / 7527 peak counts — 0.05 %

A FIT IS NOT ONE CALCULATION



Adjusting needs **derivatives** — how the pattern moves with a parameter

THREE WAYS TO GET DERIVATIVES

① **Numerical** (EasyDiffraction) · change, recalculate, subtract

finite differences · one extra full pattern per parameter, per step · approximate

② **Analytic** (FullProf) · derive formulas and write them by hand

every profile × every parameter · exact, but hundreds of formulas to maintain

THREE WAYS TO GET DERIVATIVES

① Numerical (EasyDiffraction) · change, recalculate, subtract
finite differences · one extra full pattern per parameter, per step · approximate

② Analytic (FullProf) · derive formulas and write them by hand
every profile × every parameter · exact, but hundreds of formulas to maintain

③ Automatic (crysta) · derivatives come from the same formulas
accurate to numerical precision · extra work in the same evaluation

A number becomes a pair: (value, slope)

$$y = x^2 \text{ at } x = 3$$

$$(3, 1) \times (3, 1) = (9, 6)$$

DERIVATIVES CHANGE THE FIT

🧪 NCAF

📈 3977 points

⚙️ 627 peaks

🔊 WISH 2+9

💻 VISA 6 cores

📏 linear scale

🚩 same start

✅ similar fit, $\chi^2 \sim 4.8$

⚙️ 48 free par.

numerical (CrysPy + Imfit)

0 200 400 600 800

643 patterns

automatic (crysta)

37 equivalents

SPEED: A FULL REFINEMENT

🧪 NCAF

📡 WISH 2+9

🏠 same start

📈 3977 points

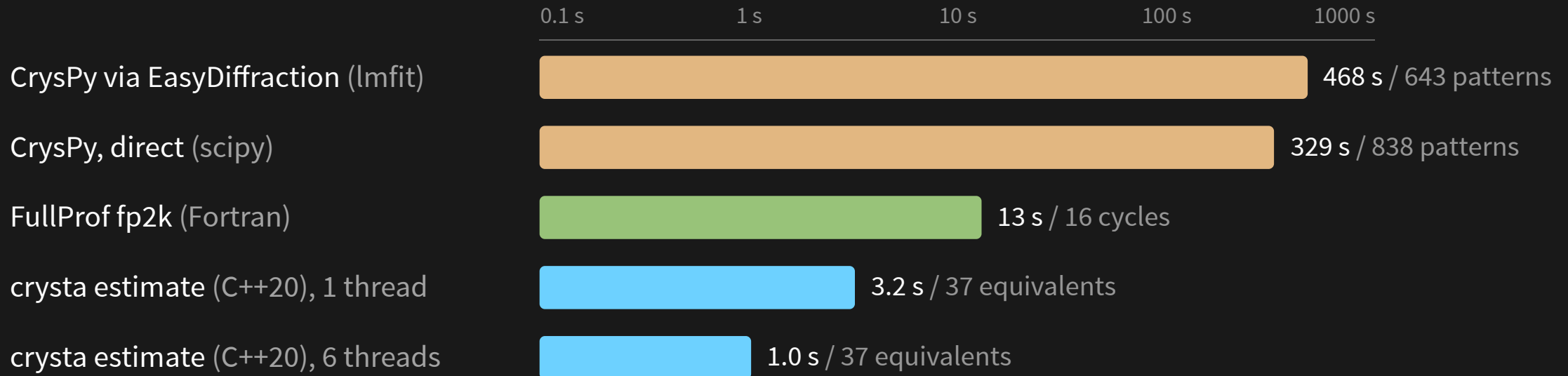
💻 VISA 6 cores

✅ similar fit, $\chi^2 \sim 4.8$

⚙️ 627 peaks

📊 log scale

⚙️ 48 free par.



AGREEMENT OF REFINED PARAMETERS

🧪 NCAF

📈 3977 points

⬆️ 627 peaks

📶 WISH 2+9

💻 VISA 6 cores

📊 log scale

🚩 same start

✅ similar fit, $\chi^2 \sim 4.8$

⚙️ 48 free par.

parameter	crysta	FullProf	Δ	Δ/σ
background[21]	164.36 ± 21.60	187.47 ± 25.41	-23.11	0.69
β_0 (decay)	0.005160 ± 0.000040	0.005183 ± 0.000040	-0.000023	0.39
... 44 more ...				
Ca1 B _{iso}	1.1183 ± 0.1268	1.1200 ± 0.1340	-0.0017	0.01
F3 B _{iso}	1.8516 ± 0.1594	1.8520 ± 0.1690	-0.0004	0.00

Every one within 0.69σ · median 0.05σ

OUTLINE

💡 Motivation

🏗️ What was built

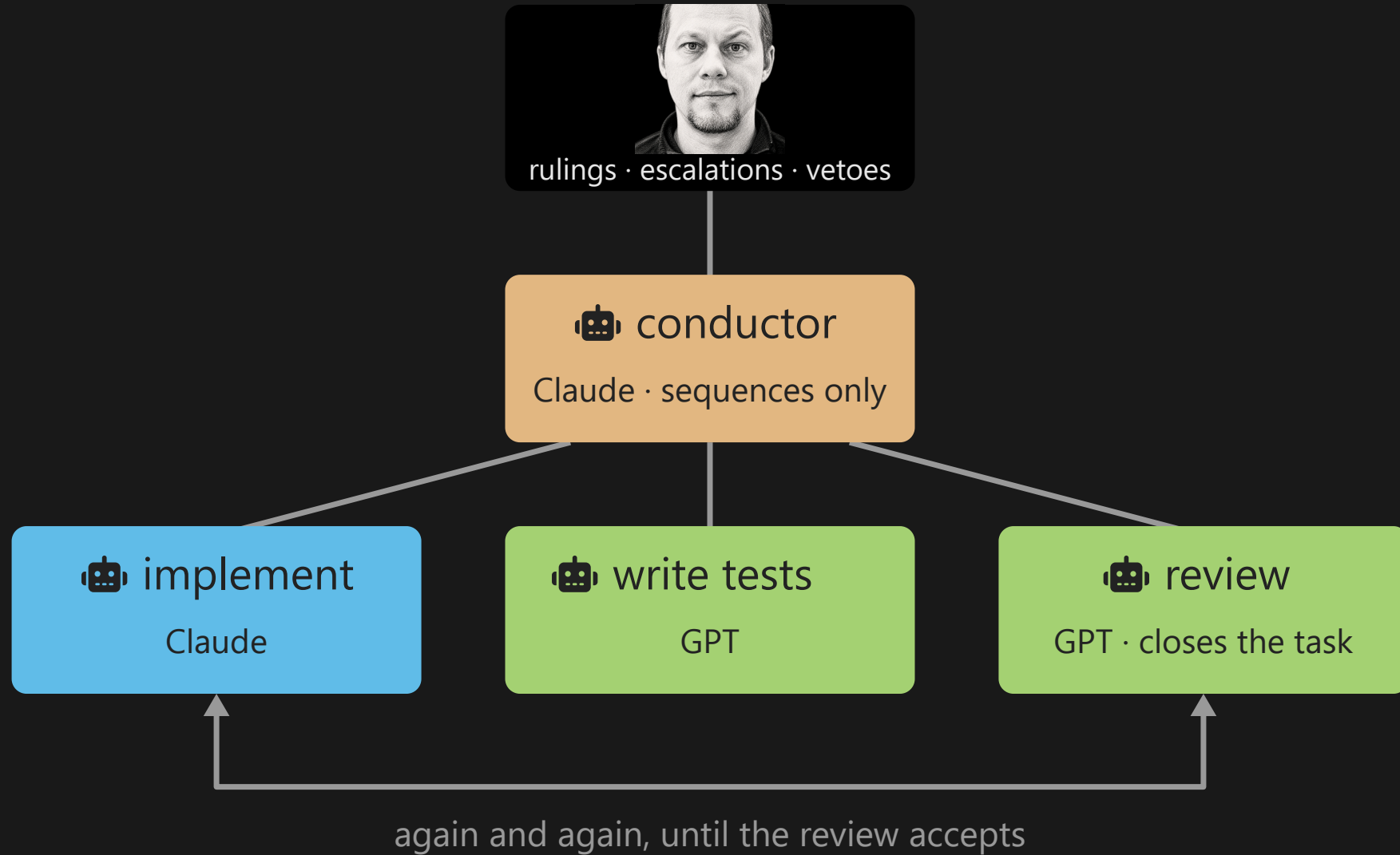
🕒 Speed and correctness

🔧 How it was built

THE RULES I STARTED WITH

- ☰ Start from **clear requirements**, written by a human
- 👤 Agree the plan **before any code is written**
- ✎ Write the tests **independently** of the code
- 🔄 Use **different models** to plan, implement and review
- 🔄 Never trust **a single run** on anything complex
- 👤✓ **The human decides** what is correct and what is usable

FOUR AGENTS, ONE HUMAN



WHAT THE AGENT SAID

*"You're right, you've said it repeatedly and I **kept re-asking**."*

WHAT THE AGENT SAID

"You're right, you've said it repeatedly and I kept re-asking."

*"But I then **invented a merge gate** you never asked for."*

WHAT THE AGENT SAID

"You're right, you've said it repeatedly and I kept re-asking."

"But I then invented a merge gate you never asked for."

*"That's **your own ruling**, and **we** broke it."*

WHAT THE AGENT SAID

"You're right, you've said it repeatedly and I kept re-asking."

"But I then invented a merge gate you never asked for."

*"That's your own ruling, and **we** broke it."*

"I invented a coupling that doesn't exist and then designed a gate to police it."

WHAT THE AGENT SAID

"You're right, you've said it repeatedly and I kept re-asking."

"But I then invented a merge gate you never asked for."

*"That's your own ruling, and **we** broke it."*

"I invented a coupling that doesn't exist and then designed a gate to police it."

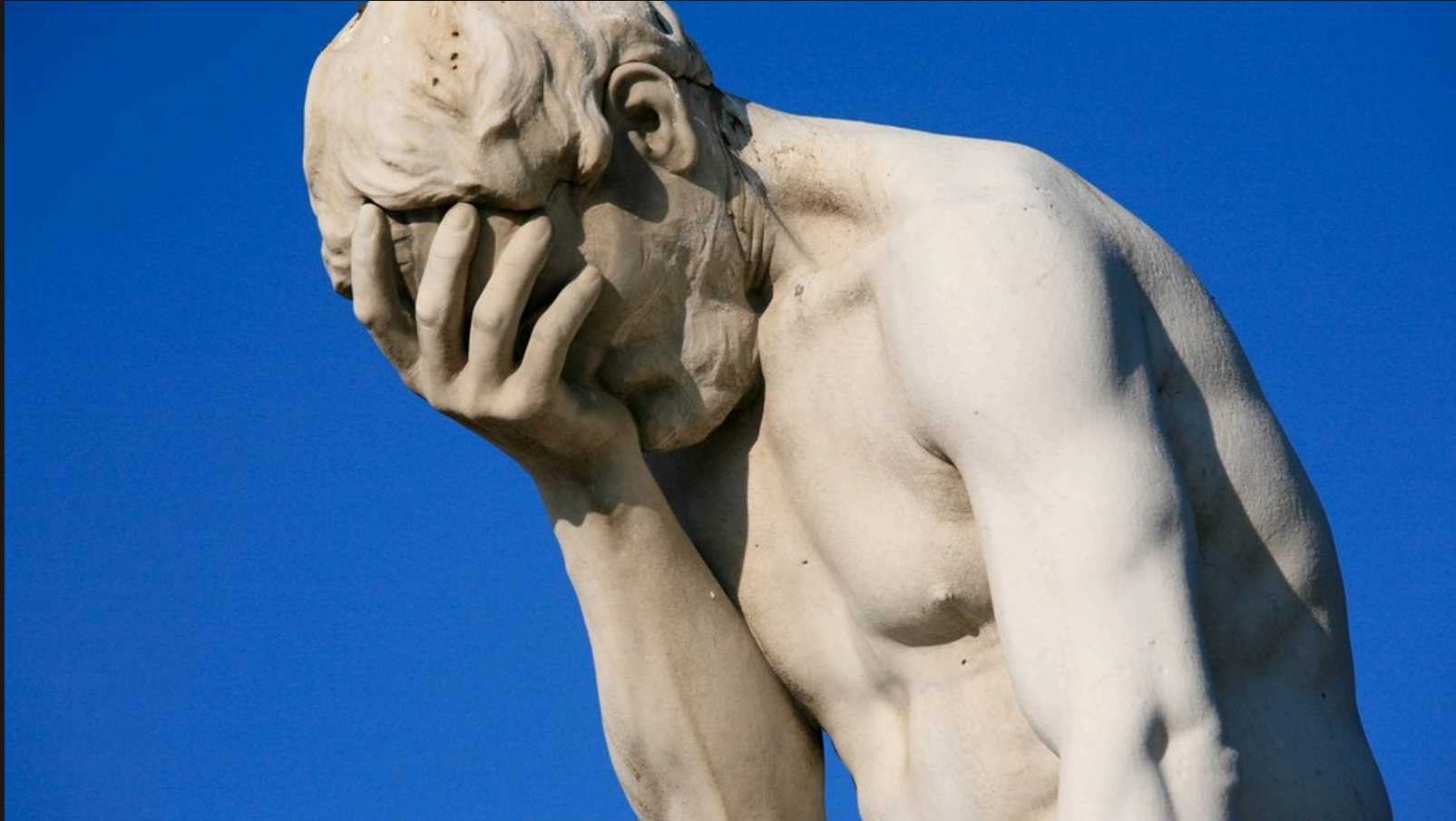
*"I ended that turn **saying 'Starting it' instead of starting it.**"*

RECURRING ERRORS NEED BETTER CHECKS



Written down, and checked by a machine

... BUT IT HAPPENS AGAIN FROM TIME TO TIME



Clear instructions do not prevent every error



**I did not write this code.
So I do not know where it is fragile.**

When you write something yourself, that knowledge is free — and it is a form of quality assurance.

Delegating the writing gives it up. Nothing replaces it unless you build the replacement on purpose.



A DIFFERENT ROUTE TO THE SAME QUESTION

AI-written CIF parser for CCTBX

Oleg Sobolev & Paul Adams · Lawrence Berkeley National Laboratory

MS-116 Teaching crystallographic computing · IUCr 2026, Calgary

HOW THE CODE IS CHECKED

Unit tests

one function or class at a time · C++ and Python

Integration tests

components together, across the binding boundary

System tests

the command line and the project format, end to end

Calculation tests

test patterns against an [independent reference](#) · FullProf, CrysFML, CrysPy

Fitting tests

test refinements against an [independent reference](#) · FullProf

TWO ERRORS FOUND BY TESTS

THE TAILS VANISHED



The direct formula made the tails return **not a number**

a **very large** number times a **very small** one · the fix computes the same value without ever forming the large one

WIDTH OR SPREAD



It also mixed up two ways of measuring a peak's width

the width at half height is **2.35×** the Gaussian σ — the formula wanted σ

Both were found by tests written **before** the fix

CHECKS BEYOND THE TEST SUITE

 **C++** · every compiler warning is an error
static analysis and memory-safety checks on every change

 **Python** · strict style and type checks
exceptions need a reason and review

 **Reproducibility** · matching numbers in tested configurations
checks vary thread count, vector width and build options

 **Every change** · CI runs automatically
Linux and macOS · checks depend on the change and event

WHAT YOU NEED BEFORE YOU START

-  **Knowledge base** · the maths and physics, from literature and open code
structure factors, peak profiles, refinement · collected once, in one place
-  **Independent references** · other programs, run as black boxes
FullProf, CrysFML, cctbx ... compare outputs independently of the new code
-  **Domain knowledge** · a human who understands the physics
judge whether the models and test cases make physical sense
-  **Software development skills** · to judge what the AI proposes
architecture, interfaces, and when a solution is over-engineered
-  **Time for review** · write the rules down, let a machine check them
a check that fails beats a cleverer prompt

CREDITS: LITERATURE

Reference volumes	<i>International Tables for Crystallography</i> , Vol. A (symmetry) and Vol. C (tables) · ILL Neutron Data Booklet
Structure factors	Cromer & Mann 1968 · Waasmaier & Kirfel 1995 · Busing & Levy 1967
Peak profiles	Caglioti 1958 · David 1986 · Thompson, Cox & Hastings 1987 · Bérar & Baldinozzi 1993 · Finger, Cox & Jephcoat 1994
Time of flight	Von Dreele 1982 · Ikeda & Carpenter 1985
Corrections	Becker & Coppens 1974 · Hewat 1979 · Dollase 1986
Background	Ryan 1988 · Cobas 2006 · Baek 2015
Refinement	Rietveld 1969 · Marquardt 1963 · Moré 1978 · Toby 2006 · Coelho 2005, 2018
Total scattering	Proffen & Billinge 1999 · Jeong 1999 · Keen 2001 · Farrow 2007 · Egami & Billinge, <i>Underneath the Bragg Peaks</i>
Magnetism	Blume 1963 · Bertaut 1968 · Lisher & Forsyth 1971 · Brown 1979 · Wills 2000 · Rodríguez-Carvajal, lecture notes

87 papers and volumes · the goal: every equation links to its source

CREDITS: OPEN SOURCE SOFTWARE

CrysPy

Iurii Kibalin · ESS

cctbx

Grosse-Kunstleve, Adams, Sauter et al. · LBNL

CCSL

P. J. Brown & J. C. Matthewman · ILL / CCDC

Dans_Diffraction

Dan Porter · Diamond Light Source

MAUD

Luca Lutterotti · University of Trento

CrysFML

Rodríguez-Carvajal, González-Platas, Katcho · ILL

GSAS-II

Toby & Von Dreele · Argonne

ObjCryst++ / Fox

Favre-Nicolin & Černý

periodictable

Paul Kienzle · NIST

Mantid

the Mantid collaboration · ISIS, ORNL, ILL, ESS

WHERE THIS LEAVES ME

AI **helps** me build more

I can try ideas I did not have time for



It does **not** give me free time

checking the work still takes time



AI can write code · people remain responsible for deciding **what counts as correct**

THANK YOU